Framework for an Optimal Execution Algorithm

July 30, 2025

1. Problem Formulation

The task is to find an optimal trading schedule, $x = [x_1, \ldots, x_N]$, for executing a total of **S** shares over **N** time intervals. The objective is to minimize the total temporary price impact, subject to the constraint that the entire order is filled.

- **Objective Function:** Minimize the total cost, $C(x)$, using the impact model $g_t(x)$ from Part 1:

$$\text{Minimize} \quad C(x) = \sum_{i=1}^{N} g_{t_i}(x_i)$$

- **Constraint:** The sum of all trades must equal the total order size:

$$\sum_{i=1}^{N} x_i = S$$

2. Proposed Solution: Dynamic Programming

This sequential decision-making problem is ideally solved using **Dynamic Programming (DP)**. DP finds the truly optimal solution by breaking the N-period problem into a series of simpler, single-period decisions and solving them recursively.

3. The Algorithm

The algorithm works by defining a "cost-to-go" function, $V(t, S_{rem})$, which is the minimum cost to execute the remaining S_{rem} shares from time t onwards. This is solved using the **Bellman Equation**:

$$V(t, S_{rem}) = \min_{0 \le x_t \le S_{rem}} [g_t(x_t) + V(t+1, S_{rem} - x_t)]$$

Here, $g_t(x_t)$ is the immediate cost of trading x_t shares, calculated using our impact model. The algorithm is implemented via **backward induction**:

1. **Base Case (t=N):** At the final step, we must trade all remaining shares, so $V(N, S_{rem}) = g_N(S_{rem})$.
2. **Inductive Step:** Starting from $t = N - 1$ and working backward to $t = 1$, we solve the Bellman equation for all possible states (t, S_{rem}), storing the optimal trade x_t^* for each.
3. **Reconstruct Schedule:** Once the process is complete, we build the optimal schedule by starting at $(1, S)$ and moving forward, selecting the stored optimal trade at each interval.

4. Integrating the Impact Model and Tools Used

The DP algorithm serves as the strategic "planner," while the impact model developed in Part 1 acts as a specialized "cost calculator." The connection between them occurs within the Bellman equation at every step. To evaluate a potential trade x_t, the framework uses a combination of tools:

1. **Feature Forecasting Model:** First, the algorithm requires a forecast of the market features (spread, imbalance, etc.) for the future time interval t. In a live system, this would be a separate time-series model. For this framework, we can simplify by assuming features remain constant.
2. **The Trained Impact Model (`g_t(x)`):** The forecasted features and the potential trade size x_t are passed as inputs to our pre-trained two-stage impact model. The model's LightGBM component predicts the liquidity parameter, $\log(\alpha_t)$, and the final cost is calculated using the full power-law formula: $g_t(x_t) = \exp(\log(\alpha_t)) \cdot (x_t^\gamma)$.
3. **Python & NumPy:** The entire DP framework, including the tables for storing costs and optimal trades, is implemented in Python, using NumPy for efficient numerical computation and array manipulation.

This DP framework provides a rigorous and computationally feasible method for determining the optimal execution schedule.